

```
In [ ]: # Initialize Otter
import otter
grader = otter.Notebook("lab2.ipynb")
```

```
In [ ]: import otter
import pandas as pd
import numpy as np
%matplotlib inline
import otter
import matplotlib.pyplot as plt
grader = otter.Notebook()
```

Fundamentos en ciencias de datos

#Laboratorio 2: Operaciones de tabla

¡Bienvenidos al Laboratorio 2! Esta semana, aprenderemos cómo importar un módulo y practicaremos operaciones de tablas. La [Referencia de Python](#) tiene información que será útil para esta práctica de laboratorio.

Lectura recomendada: * [Introducción a las Tablas](#)

Como recordatorio, estas son las políticas para obtener crédito completo:

1. Para los estudiantes inscritos en **Regular Labs** en persona, recibirán crédito de laboratorio al **asistir** a la sección de laboratorio y lograr un progreso significativo en el cuaderno de laboratorio y enviándolo a Gradescope.
2. Para los estudiantes inscritos en **Mega Lab**, recibirán crédito de laboratorio al **completar** el cuaderno, aprobar los casos de prueba y **** enviándolo**** a Gradescope antes de las **11 p. m. del viernes de la misma semana en que se publicó**.

Envío: una vez que haya terminado, ejecute todas las celdas además de la última, seleccione Archivo > Guardar cuaderno y luego ejecute la celda final. El resultado contendrá un archivo zip que podrá utilizar para enviarlo en Gradescope.

Comencemos configurando las pruebas e importaciones ejecutando la siguiente celda.

1 1. Revisión: Los componentes básicos del código Python

Los dos bloques de construcción del código Python son *expresiones* y *declaraciones*. Una **expresión** es un fragmento de código que

- es autónomo, lo que significa que tendría sentido escribirlo en una línea por sí solo, y
- generalmente se evalúa como un valor.

Aquí hay dos expresiones que se evalúan como 3:

```
3
5 - 2
```

Un tipo importante de expresión es la **expresión de llamada**. Una expresión de llamada comienza con el nombre de una función y va seguida de los argumentos de esa función entre paréntesis. La función devuelve algún valor, según sus argumentos. Algunas funciones matemáticas importantes se enumeran a continuación.

Función | Descripción |

	-----		abs		Devuelve el valor absoluto de su argumento
			máximo		Devuelve el máximo de todos sus argumentos
			min		Devuelve el mínimo de todos sus argumentos
			poder		Eleva su primer argumento a la potencia de su segundo argumento
			redondo		Redondea su argumento al entero más cercano

Aquí hay dos expresiones de llamada que se evalúan como 3:

```
abs(2 - 5)
max(round(2.8), min(pow(2, 10), -1 * pow(2, 10)))
```

La expresión `2 - 5` y las dos expresiones de llamada dadas anteriormente son ejemplos de **expresiones compuestas**, lo que significa que en realidad son combinaciones de varias expresiones más pequeñas. `2 - 5` combina las expresiones `2` y `5` mediante resta. En este caso, `2` y `5` se denominan **subexpresiones** porque son expresiones que forman parte de una expresión más grande.

Una **declaración** es una línea completa de código. Algunas declaraciones son sólo expresiones. Las expresiones enumeradas anteriormente son ejemplos.

Otras declaraciones *hacen que algo suceda* en lugar de *tener un valor*. Por ejemplo, una **declaración de asignación** asigna un valor a un nombre.

Una buena forma de pensar en esto es que estamos **evaluando el lado derecho** del signo igual y **asignándolo al lado izquierdo**. Aquí hay algunas declaraciones de asignación:

```
altura = 1.3
el_numero_cinco = abs(-5)
diferencia_altura_absoluta = abs(altura - 1.688)
```

Una idea importante en programación es que se pueden construir cosas grandes e interesantes combinando muchas cosas simples y poco interesantes. La clave para comprender un código complicado es dividirlo en sus componentes simples.

Por ejemplo, suceden muchas cosas en la última declaración anterior, pero en realidad es solo una combinación de algunas cosas. Esta imagen describe lo que está pasando.

Pregunta 1.1. En la siguiente celda, asigne el nombre `new_year` al **número mayor** entre los dos números siguientes:

- el **valor absoluto** de $2^6 - 2^{11} - 2^5 - 7$, y
- $5 \times 13 \times 31 + 7$.

Intente utilizar solo una declaración (una línea de código). Asegúrese de verificar su trabajo ejecutando la celda de prueba después.

```
In [ ]: # BEGIN SOLUTION NO PROMPT
new_year=abs(2**6-2**11-2**5-7)
# END SOLUTION

""" # BEGIN PROMPT
new_year=...
""" # END PROMPT
```

```
In [ ]: grader.check("q11")
```

Le pedimos que use una línea de código en la pregunta anterior porque solo involucra operaciones matemáticas. Sin embargo, las preguntas de programación más complicadas requerirán más pasos. No siempre es una buena idea agrupar estos pasos en una sola línea porque puede hacer que el código sea más difícil de leer y de depurar.

Una buena práctica de programación implica dividir el código en pasos más pequeños y utilizar nombres apropiados. ¡Tendrás mucha práctica en el resto de este curso!

2. Importación de código

La mayor parte de la programación implica un trabajo muy similar al trabajo que se ha realizado antes. Dado que escribir código lleva mucho tiempo, es bueno confiar en el código publicado por otros cuando sea posible. En lugar de copiar y pegar, Python nos permite **importar módulos**. Un módulo es un archivo con código Python que tiene variables y funciones definidas. Al importar un módulo, podemos usar su código en nuestro propio cuaderno.

Python incluye muchos módulos útiles que están a sólo una “importación” de distancia. Veremos el módulo “matemáticas” como primer ejemplo. El módulo “math” es extremadamente útil para calcular expresiones matemáticas en Python.

Supongamos que queremos calcular con mucha precisión el área de un círculo con un radio de 5 metros. Para eso, necesitamos la constante π , que es aproximadamente 3,14. Convenientemente, el módulo “math” tiene “pi” definido para nosotros. Ejecute la siguiente celda para importar el módulo matemático:

```
In [ ]: import math
        radius = 5
        area_of_circle = radius**2 * math.pi
        area_of_circle
```

En el código anterior, la línea “importar matemáticas” importa el módulo matemático. Esta declaración crea un módulo y luego asigna el nombre “matemáticas” a ese módulo. Ahora podemos acceder a cualquier variable o función definida en `math` escribiendo el nombre del módulo seguido de un punto, luego seguido del nombre de la variable o función que queremos.

<nombre del módulo>.<nombre>

Pregunta 2.1. El módulo `math` también proporciona el nombre `e` para la base del logaritmo natural, que es aproximadamente 2,71. Calcule $e^\pi - \pi$, dándole el nombre `near_twenty`.

Recuerda: ¡También puedes acceder a `pi` desde el módulo `math`!

```
In [1]: from math import *

        # BEGIN SOLUTION NO PROMPT
        near_twenty = e**pi-pi
        # END SOLUTION

        """ # BEGIN PROMPT
        near_twenty=...
        """ # END PROMPT

        near_twenty
```

Out[1]: 19.99909997918947

```
In [ ]: grader.check("q21")
```

2.1 2.1. Acceso a funciones

En la pregunta anterior, accedió a variables dentro del módulo “matemáticas”.

Los módulos también definen **funciones**. Por ejemplo, “matemáticas” proporciona el nombre “suelo” para la función suelo. Habiendo importado ya “matemáticas”, podemos escribir “math.floor(7.5)” para calcular el piso de 7.5. (Tenga en cuenta que la función piso devuelve el número entero más grande menor o igual a un número determinado).

Pregunta 2.1.1. Calcule el piso de pi usando floor y pi del módulo math. Asigne al resultado el nombre floor_of_pi.

```
In [4]: from math import *

        # BEGIN SOLUTION NO PROMPT
        floor_of_pi=floor(pi)

        # END SOLUTION

        """ # BEGIN PROMPT
        floor_of_pi=...
        """ # END PROMPT

        floor_of_pi
```

Out[4]: 3

```
In [ ]: grader.check("q211")
```

Para su referencia, a continuación se muestran algunos ejemplos más de funciones del módulo “math”.

Observe cómo diferentes funciones toman diferentes números de argumentos. A menudo, la [documentación](#) del módulo proporcionará información sobre cuántos argumentos se requieren para cada función.

*Sugerencia: si presiona **shift+tab** mientras está al lado de la llamada de función, aparecerá la documentación para esa función.*

```
In [ ]: # Calcular logaritmos (el logaritmo de 8 en base 2).
        # El resultado es 3 porque 2 elevado a 3 es 8.
        math.log(8, 2)
```

```
In [ ]: # Calcular raíces cuadradas.
        math.sqrt(5)
```

Hay varias formas de importar y acceder a código desde fuentes externas. El método que usamos anteriormente, `<importar <nombre_módulo>`, importa el módulo completo y requiere que usemos `<nombre_módulo>.<nombre>` para acceder a su código.

También podemos importar una constante o función específica en lugar del módulo completo. Tenga en cuenta que no es necesario utilizar el nombre del módulo de antemano para hacer referencia a ese valor en particular. Sin embargo, debe tener cuidado al reasignar los nombres de las constantes o funciones a otros valores.

```
In [ ]: # Importando solo cos y pi desde matemáticas.
        # No tenemos que usar "matemáticas" delante de cos o pi
        from math import cos, pi
        print(cos(pi))

        # Sin embargo, tenemos que usarlo delante de otras funciones matemáticas.
        math.log(pi)
```

O podemos importar cada función y valor de todo el módulo.

```
In [ ]: # Por último, podemos importar todo desde matemáticas usando el *
        # Una vez más, no tenemos que usar "matemáticas". antemano
        from math import *
        log(pi)
```

No se preocupe demasiado por qué tipo de importación utilizar. A menudo es una elección de estilo de codificación que queda en manos de cada programador. En este curso, siempre importará los módulos necesarios cuando ejecute la celda de configuración (como la primera celda de código en esta práctica de laboratorio).

¡Pasemos a practicar algunas de las operaciones de mesa que aprendiste en la conferencia!

3 3. Operaciones de mesa

La tabla `farmers_markets.csv` contiene datos sobre los mercados de agricultores en los Estados Unidos (datos asociados con [el USDA](#)). Cada fila representa uno de esos mercados.

Ejecute la siguiente celda para cargar la tabla `farmers_markets`. No habrá resultados; no se espera ningún resultado ya que la celda contiene una declaración de asignación. Una declaración de asignación no produce ningún resultado (no produce ningún valor).

```
In [ ]: # Simplemente ejecuta esta celda

farmers_markets = Table.read_table('farmers_markets.csv')
```

Examinemos nuestra tabla para ver qué datos contiene.

Pregunta 3.1. Utilice el método `show` para mostrar las primeras 5 filas de `farmers_markets`.

Nota: Los términos “método” y “función” técnicamente no son lo mismo, pero para los propósitos de este curso, los usaremos indistintamente.

Pista: `tbl.show(3)` mostrará las primeras 3 filas de la tabla denominada `tbl`. Además, asegúrese de no llamar a `.show()` sin un argumento, ya que esto bloqueará su kernel.

```
In [ ]: ...
```

Observe que faltan algunos de los valores en esta tabla, como se indica con “nan”. Esto significa que el valor no está disponible (por ejemplo, si no conocemos la dirección del mercado) o no es aplicable (por ejemplo, si el mercado no tiene una dirección). ¡También notarás que la tabla tiene una gran cantidad de columnas!

3.0.1 `num_columns`

La propiedad de la tabla `num_columns` devuelve el número de columnas de una tabla. (Una “propiedad” es solo un método al que no es necesario llamar agregando paréntesis).

Llamada de ejemplo: `tbl.num_columns` devolverá el número de columnas en una tabla llamada `tbl`

Pregunta 3.2. Utilice `num_columns` para encontrar el número de columnas en nuestro conjunto de datos de mercados de agricultores.

Asigne el número de columnas a `num_farmers_markets_columns`.

```
In [9]: import numpy as np
        from datascience import *

        farmers_markets = Table.read_table('farmers_markets.csv')

        # BEGIN SOLUTION NO PROMPT
        num_farmers_markets_columns=farmers_markets.num_columns

        # END SOLUTION

        """ # BEGIN PROMPT
        num_farmers_markets_columns= ...
        print("The table has", num_farmers_markets_columns, "columns in it!")
        """ # END PROMPT
        num_farmers_markets_columns
```

Out[9]: 59

```
In [ ]: grader.check("q32")
```

3.0.2 num_rows

De manera similar, la propiedad `num_rows` te indica cuántas filas hay en una tabla.

```
In [ ]: # Just run this cell

num_farmers_markets_rows = farmers_markets.num_rows
print("The table has", num_farmers_markets_rows, "rows in it!")
```

3.0.3 select

La mayoría de las columnas tratan sobre productos concretos: si el mercado vende tofu, comida para mascotas, etc. Si no estamos interesados en esa información, la tabla resulta difícil de leer. Esto surge más de lo que se podría pensar, porque es posible que las personas que recopilan y publican datos no sepan de antemano qué querrán hacer con ellos.

En tales situaciones, podemos usar el método de tabla “seleccionar” para elegir solo las columnas que queremos en una tabla en particular. Se necesitan muchos argumentos. Cada uno debe ser el nombre de una columna de la tabla. Devuelve una nueva tabla con solo esas columnas. Las columnas están en el orden *en el que fueron enumeradas como argumentos*.

Por ejemplo, el valor de `farmers_markets.select("MarketName", "State")` es una tabla con solo el nombre y el estado de cada mercado de agricultores en `farmers_markets`.

Pregunta 3.3. Utilice `select` para crear una tabla con solo el nombre, ciudad, estado, latitud (y) y longitud (x) de cada mercado. Llame a esa nueva tabla “`farmers_markets_locations`”.

Sugerencia: Asegúrese de ser exacto al usar nombres de columnas con `select`; ¡Compruebe dos veces las mayúsculas!

```
In [24]: import numpy as np
         from datascience import *
         farmers_markets = Table.read_table('farmers_markets.csv')

         # BEGIN SOLUTION NO PROMPT
         farmers_markets_locations=farmers_markets.select('y','x')
```

```

# END SOLUTION

""" # BEGIN PROMPT
farmers_markets_locations= ...
""" # END PROMPT
farmers_markets_locations

```

```

Out[24]: y      | x
         44.411 | -72.1403
         41.3751 | -81.7286
         42.296  | -85.5749
         34.8042 | -82.8187
         37.4956 | -94.2746
         40.7939 | -73.9493
         36.1184 | -86.7907
         40.809  | -73.9482
         39.7421 | -75.5345
         38.917  | -77.0321
         ... (8536 rows omitted)

```

```

In [ ]: grader.check("q33")

```

3.0.4 drop

`drop` tiene el mismo propósito que `select`, pero elimina las columnas que usted proporciona en lugar de las que no proporciona. Al igual que “seleccionar”, “soltar” devuelve una nueva tabla.

Pregunta 3.4. Supongamos que simplemente no desea las columnas `FMID` y `updateTime` en `farmers_markets`. Cree una tabla que sea una copia de `farmers_markets` pero que no incluya esas columnas. Llame a esa tabla “`farmers_markets__without_fmids`”.

```

In [38]: import numpy as np
         from datascience import *
         farmers_markets = Table.read_table('farmers_markets.csv')

# BEGIN SOLUTION NO PROMPT
farmers_markets_without_fmids = farmers_markets.drop('FMID', 'updateTime')

# END SOLUTION

""" # BEGIN PROMPT
farmers_markets_without_fmids = ...
""" # END PROMPT
farmers_markets_without_fmids

```

```

Out[38]: MarketName | street
         Caledonia Farmers Market Association - Danville | nan
         Stearns Homestead Farmers' Market | 6975 Ridge Road
         100 Mile Market | 507 Harrison St
         106 S. Main Street Farmers Market | 106 S. Main Street
         10th Steet Community Farmers Market | 10th Street and Poplar
         112st Madison Avenue | 112th Madison Avenue
         12 South Farmers Market | 3000 Granny White Pike
         125th Street Fresh Connect Farmers' Market | 163 West 125th Street and Adam Clayton Powe
         12th & Brandywine Urban Farm Market | 12th & Brandywine Streets
         14&U Farmers' Market | 1400 U Street NW
         ... (8536 rows omitted)

```

```
In [ ]: grader.check("q34")
```

Ahora supongamos que queremos responder algunas preguntas sobre los mercados de agricultores en Estados Unidos. Por ejemplo, ¿qué mercado(s) tiene la mayor longitud (dada por la columna “x”)?

Para responder a esto, ordenaremos `farmers_markets_locations` por longitud.

```
In [ ]: farmers_markets_locations.sort('x')
```

Vaya, eso no respondió nuestra pregunta porque ordenamos de menor a mayor longitud. Para ver las longitudes más grandes, tendremos que ordenar en orden inverso.

```
In [ ]: farmers_markets_locations.sort('x', descending=True)
```

(The `descending=True` se llama *argumento opcional*. Tiene un valor predeterminado de `False`, por lo que cuando le dices explícitamente a la función `descending=True`, la función ordenará en orden descendente).

3.0.5 sort

Algunos detalles sobre la clasificación:

1. El primer argumento para “ordenar” es el nombre de una columna por la que ordenar.
2. Si la columna tiene texto, “ordenar” ordenará alfabéticamente; si la columna tiene números, se ordenará numéricamente, ambos en orden ascendente de forma predeterminada.
3. El valor de `farmers_markets_locations.sort("x")` es una *copia* de `farmers_markets_locations`; la tabla `farmers_markets_locations` no se modifica. Por ejemplo, si llamamos a `farmers_markets_locations.sort("x")`, ejecutar `farmers_markets_locations` por sí solo aún devolvería la tabla sin clasificar.

4. Las filas siempre permanecen juntas cuando se ordena una tabla. No tendría sentido ordenar solo una columna y dejar las otras columnas como están. Por ejemplo, en este caso, si ordenáramos solo la columna “x”, todos los mercados de agricultores terminarían con las longitudes incorrectas.

Pregunta 3.5. Cree una versión de `farmers_markets_locations` que esté ordenada por **latitud (y)**, con las latitudes más grandes primero. Llámelo “`farmers_markets_locations_by_latitude`”.

```
In [59]: import numpy as np
         from datascience import *
         farmers_markets = Table.read_table('farmers_markets.csv')

         # BEGIN SOLUTION NO PROMPT
         farmers_markets_locations=farmers_markets.select('y','x')
         farmers_markets_locations_by_latitude=farmers_markets_locations.drop('x')
         farmers_markets_locations_by_latitude=farmers_markets_locations_by_latitude.sort('y')

         # END SOLUTION

         """ # BEGIN PROMPT
         farmers_markets_locations_by_latitude= ...
         """ # END PROMPT

         farmers_markets_locations_by_latitude
```

```
Out[59]: y
         17.7099
         17.7322
         17.7449
         17.9686
         17.9723
         17.9782
         17.9792
         17.9854
         18.0069
         18.0089
         ... (8536 rows omitted)
```

```
In [ ]: grader.check("q35")
```

Ahora digamos que queremos una tabla de todos los mercados de agricultores de California. La clasificación no nos ayudará mucho aquí porque California está más cerca del centro del conjunto de datos.

En su lugar, utilizamos el método de la tabla. `where`.

```
In [ ]: california_farmers_markets = farmers_markets_locations.where('State', are.equal_to('California'))
         california_farmers_markets
```

Ignora la sintaxis por el momento. En su lugar, intenta leer esa línea así:

Asigne el nombre `california_farmers_markets` a una tabla cuyas filas son las filas de la tabla `farmers_markets_locations` donde están los "Estados" `are equal to "California"`.

3.0.6 where

Ahora profundicemos un poco más en los detalles. `where` toma 2 argumentos:

1. El nombre de una columna. `where` busca filas donde los valores de esa columna cumplen algún criterio.
2. Un predicado que describe el criterio que debe cumplir la columna.

El predicado en el ejemplo anterior llamó a la función `are.equal_to` con el valor que queríamos, 'California'. Pronto veremos otros predicados.

`where` devuelve una tabla que es una copia de la tabla original, pero **con solo las filas que cumplen el predicado dado**.

Pregunta 3.6. Utilice `california_farmers_markets` para crear una tabla llamada `berkeley_markets` que contenga los mercados de agricultores en Berkeley, California.

```
In [84]: import numpy as np
         from datascience import *

         farmers_markets = Table.read_table('farmers_markets.csv')

         # BEGIN SOLUTION NO PROMPT
         berkeley_markets=farmers_markets.where('State', are.equal_to('California'))

         # END SOLUTION

         """ # BEGIN PROMPT
         berkeley_markets= ...
         """ # END PROMPT

         berkeley_markets
```

```
Out[84]: FMID      | MarketName                | street                | city
         1005272 | Adelanto Stadium Farmers Market | Hwy 395 & Rancho Drive | Victorville
         1009019 | Alameda Farmers' Market    | Haight and Webster St. | Alameda
         1008835 | Alisal Certified Farmers' Market | 632 East Alisal Street | Salinas
         1009998 | Altadena Farmers' Market   | 600 W. Palm Street    | Altadena
         1011421 | Alum Rock Village Farmers' Market | 57 North White Road    | San Jose
```

```

1005644 | Amador Farmers' Market-- Jackson      | Jackson Library/ True Value | Jackson
1005645 | Amador Farmers' Market-- Pine Grove    | Hwy 88 at Irishtown Rd.    | Pine Grove
1005646 | Amador Farmers' Market-- Sutter Creek   | Eureka St. & Highway 49    | Sutter Creek
1002089 | Anderson Happy Valley Farmers Market    | Happy Valley Rd.           | Anderson
1006268 | Angels Camp Farmers Market-Fresh Fridays | Utica Park, Utica Lane      | Angels Camp
... (745 rows omitted)

```

```
In [ ]: grader.check("q36")
```

Hasta ahora solo hemos estado usando **dónde** con el predicado que requiere encontrar que los valores en una columna sean *exactamente* iguales a un cierto valor. Sin embargo, hay muchos otros predicados. Hemos enumerado algunos de los más comunes a continuación, pero si desea una lista más completa, consulte la sección de predicados en la [hoja de referencia de Python] (<https://www.data8.org/fa23/reference/#tablewhere-predicates>).

Predicate	Example	Result
<code>are.equal_to</code>	<code>are.equal_to(50)</code>	Find rows with values equal to 50
<code>are.not_equal_to</code>	<code>are.not_equal_to(50)</code>	Find rows with values not equal to 50
<code>are.above</code>	<code>are.above(50)</code>	Find rows with values above (and not equal to) 50
<code>are.above_or_equal_to</code>	<code>are.above_or_equal_to(50)</code>	Find rows with values above 50 or equal to 50
<code>are.below</code>	<code>are.below(50)</code>	Find rows with values below 50
<code>are.between</code>	<code>are.between(2, 10)</code>	Find rows with values above or equal to 2 and below 10
<code>are.between_or_equal_to</code>	<code>are.between_or_equal_to(2, 10)</code>	Find rows with values above or equal to 2 and below or equal to 10

3.1 4. Analizando un conjunto de datos

Ahora que está familiarizado con las operaciones de tablas, ¡respondamos una pregunta interesante sobre un conjunto de datos!

Ejecute la siguiente celda para cargar la tabla `imdb`. Contiene información sobre las 250 películas mejor calificadas en IMDb.

```

In [ ]: # Simplemente ejecuta esta celda

imdb = Table.read_table('imdb.csv')
imdb

```

A menudo, queremos realizar múltiples operaciones (clasificar, filtrar u otras) para convertir una tabla que tenemos en algo más útil. Puede realizar estas operaciones una por una, por ejemplo.

```
first_step = original_tbl.where("col1", are.equal_to(12))
second_step = first_step.sort('col2', descending=True)
```

Sin embargo, dado que el valor de la expresión `original_tbl.where("col1", are.equal_to(12))` es en sí mismo una tabla, puedes simplemente llamar a un método de tabla en él:

```
original_tbl.where("col1", are.equal_to(12)).sort('col2', descending=True)
```

Debe organizar su trabajo de la manera que tenga más sentido para usted, utilizando nombres informativos para las tablas intermedias que cree.

Pregunta 4.1. Cree una tabla de películas estrenadas entre 2010 y 2015 (inclusive) con calificaciones superiores a 8. La tabla solo debe contener las columnas `Title` and `Rating`, **en ese orden**.

Asigne la tabla al nombre `above_eight`.

Pista: Piensa en los pasos que debes seguir e intenta ordenarlos en un orden que tenga sentido. ¡Siéntete libre de crear tablas intermedias para cada paso, pero asegúrate de asignar a tu tabla final el nombre “`above_eight`”!

```
In [ ]: import numpy as np
        from datascience import *

        imdb = Table.read_table('imdb.csv')

        # BEGIN SOLUTION NO PROMPT
        above_eigh=imdb.where('Rating',are.above_or_equal_to(8)).where('Year',are.between_or_equal_to(2010,2015))

        # END SOLUTION

        """ # BEGIN PROMPT
        above_eigh= ...
        """ # END PROMPT

        above_eigh

In [ ]: grader.check("q41")
```

Pregunta 4.2. Utilice `num_rows` (y aritmética) para encontrar la *el numero* de películas en el conjunto de datos que se estrenaron entre 1900 y 1999, y la *el numero* de películas en el conjunto de datos que se estrenaron en el año 2000 o después.

Asigne “num_en_siglo_20” a el número de películas en el conjunto de datos que se estrenaron entre 1900 y 1999, y “num_en_siglo_21” a el número de películas en el conjunto de datos que se estrenaron en el año 2000 o después.

Pista: El *número* de películas estrenadas en el siglo XX es el *número* de películas estrenadas en el siglo XX, dividida por el *número total* de películas.

```
In [1]: import numpy as np
        from datascience import *

        imdb = Table.read_table('imdb.csv')

        # BEGIN SOLUTION NO PROMPT

        num_movies_in_dataset=imdb.num_rows
        num_in_siglo_20=imdb.where('Year',are.between_or_equal_to(1900, 1999))
        num_in_21st_century=imdb.where('Year',are.above_or_equal_to(2000))

        proportion_in_20th_century=num_in_siglo_20.num_rows/num_movies_in_dataset
        proportion_in_21st_century=num_in_21st_century.num_rows/num_movies_in_dataset

        # END SOLUTION

        """ # BEGIN PROMPT
        num_movies_in_dataset = ...
        num_in_20th_century = ...
        num_in_21st_century = ...
        proportion_in_20th_century = ...
        proportion_in_21st_century = ...
        """ # END PROMPT

        proportion_in_20th_century
        #proportion_in_21st_century
```

```
/usr/lib/python3/dist-packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.17.3 and <1.25.0
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")
```

```
Out[1]: 0.684
```

```
In [ ]: grader.check("q42")
```

3.2 5. Resumen

Para su referencia, aquí hay una tabla de todas las funciones y métodos que vimos en esta práctica de laboratorio. ¡Aprenderemos más métodos para agregar a esta tabla la próxima semana!

Nombre	Ejemplo	Propósito
<code>sort</code>	<code>tbl.sort("N")</code>	Crear una copia de una tabla ordenada por los valores de una columna
<code>where</code>	<code>tbl.where("N", are.above(2))</code>	Crea una copia de una tabla con solo las filas que coinciden con algún <i>predicado</i>
<code>num_rows</code>	<code>tbl.num_rows</code>	Calcular el número de filas en una tabla
<code>num_columns</code>	<code>tbl.num_columns</code>	Calcular el número de columnas en una tabla
<code>select</code>	<code>tbl.select("N")</code>	Crear una copia de una tabla con solo algunas de las columnas
<code>drop</code>	<code>tbl.drop("N")</code>	Crear una copia de una tabla sin algunas de las columnas

¡Lego quiere felicitarte por terminar el Laboratorio 2!

¡Terminaste con el laboratorio!

Información importante sobre el envío: - Ejecuta todas las pruebas y verifica que todas pasan - Guardar desde el menú Archivo - Ejecute la celda final para generar el archivo zip - Haga clic en el enlace para descargar el archivo zip - Luego, ve a [Gradescope](#) y envía el archivo zip a la tarea correspondiente. El nombre de esta tarea es “Lab XX Autograder”, donde XX es el número de laboratorio: 01, 02, 03, etc.

- Si terminas temprano en el laboratorio regular, **pídele a uno de los miembros del personal que te marque.**

Es su responsabilidad asegurarse de que su trabajo esté guardado antes de ejecutar la última celda.

3.3 Envío

Asegúrese de haber ejecutado todas las celdas de su cuaderno en orden antes de ejecutar la siguiente celda, para que todas las imágenes/gráficos aparezcan en el resultado. La siguiente celda generará un archivo zip para que lo envíe. **¡Guarde antes de exportar!**

```
In [ ]: # Save your notebook first, then run this cell to export your submission.  
        grader.export(pdf=False, run_tests=True)
```

3.4 Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a zip file for you to submit. **Please save before exporting!**

These are some submission instructions.

```
In [ ]: # Save your notebook first, then run this cell to export your submission.  
        grader.export(run_tests=True)
```